

Basic C Without Tears

Guía Práctica con Soluciones – Tipo A

Fundamentos de C, Control de Flujo, Arreglos y Cadenas de Caracteres

Autor: Stack

1. Responda a las siguientes preguntas teóricas y prácticas sobre memoria, tipos de datos primitivos y operadores.

- (a) Indique si la siguiente afirmación es Verdadera o Falsa. Justifique detallando el cálculo exacto de memoria en bytes (asuma una arquitectura estándar donde `sizeof(int) = 4`, `sizeof(double) = 8` y `sizeof(char) = 1`):

«Un arreglo `double medidas[200]` ocupa en la memoria física exactamente la misma cantidad total de bytes que una matriz bidimensional `int matriz[100][4]` y que un arreglo `char buffer[1600]`».

Solución y Explicación:

Respuesta: Verdadero.

- `double medidas[200]`: $200 \times \text{sizeof}(\text{double}) = 200 \times 8 = 1600$ bytes.
- `int matriz[100][4]`: $100 \times 4 \times \text{sizeof}(\text{int}) = 400 \times 4 = 1600$ bytes.
- `char buffer[1600]`: $1600 \times \text{sizeof}(\text{char}) = 1600 \times 1 = 1600$ bytes.

Las tres estructuras reservan exactamente la misma cantidad de memoria contigua en el Stack (1600 bytes).

- (b) En C, los caracteres se almacenan internamente como números enteros pequeños según la tabla ASCII. Explique detalladamente la salida impresa por el siguiente fragmento de código, justificando por qué la operación `objetivo - base` es válida y cómo opera la transformación aritmética a mayúscula:

```
char base = 'a';
char objetivo = 'd';
int distancia = objetivo - base;
char mayuscula = base - 32;

printf("Distancia: %d, Mayuscula: %c\n", distancia, mayuscula);
```

Solución y Explicación:

Salida por pantalla: Distancia: 3, Mayuscula: A

Análisis conceptual:

- 1) El tipo `char` almacena enteros de 8 bits según la tabla ASCII. En dicha codificación, 'a' equivale a 97 y 'd' equivale a 100.
- 2) La expresión `objetivo - base` ejecuta una resta regular entre enteros ($100 - 97 = 3$), representando la distancia relativa entre ambas letras en el abecedario.
- 3) En ASCII, las letras mayúsculas se encuentran 32 posiciones antes de sus correspondientes minúsculas ('A' es 65, 'a' es 97). Al restar 32 al código de 'a' se obtiene 65, que con `%c` se renderiza como 'A'.

- (c) El siguiente programa tiene como objetivo calcular el costo unitario prorrateado de un lote de producción, pero al ejecutarse imprime en consola un resultado erróneo de \$0.00:

```
1 int total_gastos = 450;
2 int unidades = 1000;
3 float costo_unitario;
4
5 costo_unitario = total_gastos / unidades;
6 printf("Costo por unidad: $%.2f\n", costo_unitario);
```

Identifique con precisión el error conceptual de tipos que provoca este comportamiento y escriba la instrucción de asignación corregida.

Solución y Explicación:

Error conceptual: Tanto `total_gastos` como `unidades` son variables de tipo `int`. En C, cuando ambos operandos son enteros, el operador `/` realiza una **división entera**, truncando inmediatamente la parte decimal a cero ($450/1000 = 0$). Posteriormente, ese entero 0 es promovido a `0.0f` durante la asignación a la variable flotante, perdiéndose la precisión decimal antes de almacenarse.

Instrucción corregida: Se debe forzar una división en punto flotante aplicando conversión explícita (*type casting*) a cualquiera de los operandos:

```
costo_unitario = (float)total_gastos / unidades;
```

2. Una empresa de logística y distribución de paquetería evalúa los paquetes que ingresan a su centro de distribución según su peso P (en kilogramos) y sus dimensiones: largo L , ancho W y alto H (en centímetros).

- (a) Escriba el fragmento condicional en C (**if-else**) que determine si un paquete es admisible para su transporte. Un paquete debe ser **rechazado** si su peso excede los 30,0 kg, si cualquiera de sus dimensiones individuales (L, W o H) supera los 120,0 cm, o si la suma de las tres dimensiones ($L + W + H$) supera los 250,0 cm. En caso contrario, el paquete es **aceptado**.

Solución y Explicación:

Se unen las condiciones de rechazo mediante el operador lógico disyunción (**||**):

```
1 // Evaluacion de condiciones de admision
2 if (peso > 30.0f || largo > 120.0f || ancho > 120.0f || alto > 120.0f ||
    (largo + ancho + alto) > 250.0f) {
3     printf("Paquete rechazado: excede los limites permitidos de peso o
        tamano.\n");
4 } else {
5     printf("Paquete aceptado para distribucion.\n");
6 }
```

- (b) Si el paquete es aceptado, la tarifa base no se calcula siempre por el peso en báscula, sino sobre el *peso facturable*, el cual se define formalmente como el valor máximo entre el peso real y el peso volumétrico (PV), calculado como:

$$PV = \frac{L \times W \times H}{5000,0}$$

Escriba las instrucciones en C que calculen PV y asignen a la variable `peso_facturable` el mayor de ambos valores, utilizando el operador ternario (`?:`) o una estructura condicional.

Solución y Explicación:

Es fundamental utilizar un literal de punto flotante (5000.0f) en el denominador para evitar la división entera:

```
1 // Calculo del peso volumetrico
2 float peso_vol = (largo * ancho * alto) / 5000.0f;
3
4 // Seleccion del mayor entre peso real y volumetrico
5 float peso_facturable = (peso > peso_vol) ? peso : peso_vol;
6
7 printf("Peso real: %.2f kg | Peso volumetrico: %.2f kg\n", peso,
    peso_vol);
8 printf("Peso facturable aplicado: %.2f kg\n", peso_facturable);
```

(c) Para calcular el cobro final en pesos (\$) se aplican las siguientes reglas:

- Tarifa base de \$3.000 para paquetes de hasta 2,0 kg de peso facturable.
- Para pesos facturables superiores a 2,0 kg, se cobran los \$3.000 base más \$1.200 por cada kilo (o fracción continua) excedente.
- Si la variable entera `es_zona_remota` tiene el valor 1, se aplica un recargo adicional del 25 % sobre el costo total acumulado.

Escriba el fragmento de código que calcule e imprima el total a pagar formateado como número entero sin decimales.

Solución y Explicación:

```
1 float costo = 3000.0f;
2
3 // Cobro de excedente para paquetes de mas de 2.0 kg
4 if (peso_facturable > 2.0f) {
5     costo += (peso_facturable - 2.0f) * 1200.0f;
6 }
7
8 // Recargo adicional por zona remota (25%)
9 if (es_zona_remota == 1) {
10     costo *= 1.25f;
11 }
12
13 printf("Costo total de envio: $%.0f\n", costo);
```

3. En sistemas de telemetría y monitoreo de sensores industriales, las lecturas continuas se almacenan en un arreglo unidimensional `int A[n]`. Se define formalmente como un **pico local** a cualquier elemento que sea estrictamente mayor que sus dos vecinos contiguos (inmediato anterior e inmediato posterior).

- (a) Explique por qué la condición de pico local no puede evaluarse con la misma expresión en los extremos del arreglo ($i = 0$ e $i = n - 1$). ¿Qué error de memoria ocurre a bajo nivel en C si se ejecuta la comparación `A[i] > A[i-1]` en el índice $i = 0$?

Solución y Explicación:

Los elementos extremos carecen de un par completo de vecinos contiguos: en $i = 0$ no existe un vecino anterior y en $i = n - 1$ no existe un vecino posterior.

Al evaluar `A[i-1]` con $i = 0$, el programa genera el índice no válido `A[-1]`, intentando acceder a una dirección de memoria ubicada 4 bytes (`sizeof(int)`) antes del inicio del arreglo. Dado que C no comprueba límites (*no bounds checking*), se produce un desbordamiento que lee basura o provoca una caída por *Segmentation Fault*. Por ello, la iteración debe restringirse a los elementos estrictamente interiores: $1 \leq i \leq n - 2$.

- (b) Para el arreglo de lecturas `int V[7] = {12, 18, 14, 14, 22, 9, 15};`, realice un trazado manual de los índices interiores ($i = 1, 2, 3, 4, 5$). Indique explícitamente cuáles de ellos cumplen con la condición de pico local y determine la cantidad total de picos detectados.

Solución y Explicación:

Evaluación secuencial de cada posición interior:

- **i = 1** ($V[1] = 18$): Vecinos son $V[0] = 12$ y $V[2] = 14$. ¿ $18 > 12$ y $18 > 14$? **Sí** → **Pico local 1.**
- **i = 2** ($V[2] = 14$): Vecinos son $V[1] = 18$ y $V[3] = 14$. ¿ $14 > 18$? **No** → No califica.
- **i = 3** ($V[3] = 14$): Vecinos son $V[2] = 14$ y $V[4] = 22$. ¿ $14 > 14$? **No** (debe ser estrictamente mayor) → No califica.
- **i = 4** ($V[4] = 22$): Vecinos son $V[3] = 14$ y $V[5] = 9$. ¿ $22 > 14$ y $22 > 9$? **Sí** → **Pico local 2.**
- **i = 5** ($V[5] = 9$): Vecinos son $V[4] = 22$ y $V[6] = 15$. ¿ $9 > 22$? **No** → No califica.

Total detectado: Existen exactamente **2 picos locales** (en los índices 1 y 4, con valores de 18 y 22).

- (c) Escriba un programa completo o función en C que reciba un arreglo `int A[n]`, detecte todos los picos locales interiores, los almacene en un arreglo auxiliar `picos` y reporte por consola la cantidad total y los valores encontrados. Considere los siguientes casos de prueba:

Entrada (A[n])	Salida esperada
{5, 10, 4, 8, 2, 9, 3}	Picos encontrados (3): 10 8 9
{1, 2, 3, 4, 5}	Picos encontrados (0): ninguno
{10, 4, 7, 7, 7, 3, 8}	Picos encontrados (0): ninguno

Solución y Explicación:

```
1 #include <stdio.h>
2
3 int main(void) {
4     int A[] = {5, 10, 4, 8, 2, 9, 3};
5     int n = sizeof(A) / sizeof(A[0]);
6     int picos[n];
7     int total_picos = 0;
8
9     // Recorrido seguro de los elementos interiores (1 a n-2)
10    for (int i = 1; i < n - 1; i++) {
11        if (A[i] > A[i - 1] && A[i] > A[i + 1]) {
12            picos[total_picos] = A[i];
13            total_picos++;
14        }
15    }
16
17    // Reporte de resultados
18    if (total_picos > 0) {
19        printf("Picos encontrados (%d): ", total_picos);
20        for (int j = 0; j < total_picos; j++) {
21            printf("%d ", picos[j]);
22        }
23        printf("\n");
24    } else {
25        printf("Picos encontrados (0): ninguno\n");
26    }
27
28    return 0;
29 }
```

4. En C, las cadenas de caracteres (strings) se implementan como arreglos de tipo `char` delimitados por el carácter nulo terminador `'\0'`.

(a) Responda a las siguientes preguntas conceptuales:

1. ¿Cuál es la cantidad mínima de bytes que debe reservarse en memoria para almacenar de forma segura la palabra "SISTEMAS" como una cadena de caracteres en C? Justifique.
2. ¿Cuál es el propósito del carácter `'\0'` (cero binario) y qué consecuencia tiene en la ejecución de funciones como `printf("%s", ...)` si este carácter se omite en memoria?

Solución y Explicación:

- 1) **Reserva mínima de memoria:** Se requieren exactamente **9 bytes**. Aunque la palabra "SISTEMAS" consta de 8 caracteres útiles, en C toda cadena terminada en nulo necesita un byte adicional para almacenar el carácter centinela `'\0'`. Si se reservan solo 8 bytes, la secuencia no constituye un string válido según la especificación de C.
- 2) **Función de `'\0'`:** C no almacena la longitud de un string junto con la variable. Las funciones estándar como `printf("%s", ...)` o `strlen` comienzan a leer desde la dirección de inicio e imprimen carácter por carácter de forma continua hasta encontrar un byte con valor numérico 0 (`'\0'`). Si este terminador no existe, la función continúa leyendo fuera del buffer reservado, imprimiendo basura de memoria y provocando vulnerabilidades de acceso no autorizado o caídas por *Segmentation Fault*.

- (b) Un estudiante implementó la siguiente función con el objetivo de invertir una cadena de caracteres en su mismo espacio de memoria (*in-place*), pero al probarla la cadena resultante queda inalterada (idéntica a la original):

```
1 void invertir(char str[], int len) {  
2     for (int i = 0; i < len; i++) {  
3         char temp = str[i];  
4         str[i] = str[len - 1 - i];  
5         str[len - 1 - i] = temp;  
6     }  
7 }
```

Explique por qué falla esta lógica algorítmica y escriba la corrección precisa que debe aplicarse en la condición de parada del ciclo `for`.

Solución y Explicación:

Explicación del fallo: El bucle recorre todos los índices de 0 hasta $len - 1$. En la primera mitad ($i < len/2$), los caracteres se intercambian correctamente; sin embargo, al cruzar el punto medio ($i \geq len/2$), la iteración vuelve a intercambiar los caracteres simétricos ya movidos, regresando la cadena a su disposición original.

Corrección: El bucle debe detenerse estrictamente en la mitad del arreglo:

```
for (int i = 0; i < len / 2; i++)
```

(c) Escriba una función en C con el siguiente prototipo:

```
void generar_acronimo(const char frase[], char acronimo[]);
```

La función debe procesar la cadena `frase` y construir en `acronimo` las iniciales en mayúscula de cada palabra. Se asume que una nueva palabra comienza al inicio de la frase o tras uno o más espacios en blanco consecutivos (' '). Asegure la correcta colocación del carácter terminador `'\0'`.

Entrada (frase)	Salida (acronimo)
"basic c without tears"	"BCWT"
"lenguaje c estandar "	"LCE"
"memoria"	"M"

Solución y Explicación:

```
1 void generar_acronimo(const char frase[], char acronimo[]) {
2     int i = 0;
3     int k = 0;
4     int nueva_palabra = 1; // Bandera de estado para detectar el inicio
                           // de palabra
5
6     while (frase[i] != '\0') {
7         if (frase[i] != ' ') {
8             if (nueva_palabra) {
9                 char c = frase[i];
10                // Si la letra es minuscula ('a'..'z'), convertir a
11                // mayuscula restando 32
12                if (c >= 'a' && c <= 'z') {
13                    c = c - 32;
14                }
15                acronimo[k] = c;
16                k++;
17                nueva_palabra = 0; // Ya se capturo la inicial de la
18                // palabra en curso
19            }
20            else {
21                // El espacio habilita la captura de la proxima inicial
22                nueva_palabra = 1;
23            }
24            i++;
25        }
26        // Cierre indispensable del string generado con terminador nulo
27        acronimo[k] = '\0';
28    }
```